AI-Assisted-Coding Week-13.1

2303A51678

B-28

Task Description #1 (Refactoring – Removing Code Duplication)

Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

Instructions:

- Prompt AI to identify duplicate logic and replace it with functions or classes.
- Ensure the refactored code maintains the same output. Add docstrings to all functions.

CODE:

```
#Legacy script with repeated logic
#print("Area of Rectangle:", 5 * 10)
#print("Perimeter of Rectangle:", 2 * (5 + 10))
#print("Area of Rectangle:", 7 * 12)
#print("Perimeter of Rectangle:", 2 * (7 + 12))
#print("Area of Rectangle:", 10 * 15)
#print("Perimeter of Rectangle:", 2 * (10 + 15))
#Refactored code with a reusable function and no duplication.

def calculate_rectangle_properties(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
calculate_rectangle_properties(5, 10)
calculate_rectangle_properties(7, 12)
calculate_rectangle_properties(10, 15)
```

OUTPUT:

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

Task: Use AI to analyze a Python script with nested loops and complex conditionals.

Instructions:

- Ask AI to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).
- Implement changes while keeping logic intact. □ Compare execution time before and after refactoring.

CODE:

```
# Legacy inefficient code
#names = ["Alice", "Bob", "Charlie", "David"]
#search_names = ["Charlie", "Eve", "Bob"]
#for s in search_names:
#found = False
#for n in names:
#if s == n:
#found = True
#if found:
#print(f"{s} is in the list")
#else:
#print(f"{s} is not in the list")
#Optimized code using set lookups with performance comparison table.
names = {"Alice", "Bob", "Charlie", "David"}
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
    if s in names:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

OUTPUT:

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Task Description #3 (Refactoring – Extracting Reusable Functions)

Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

Instructions:

- Identify repeated or related logic and extract it into reusable functions.
- Ensure the refactored code is modular, easy to read, and documented with docstrings.

CODE:

```
# Legacy script with inline repeated logic
#price = 250
#tax = price * 0.18
#total = price + tax
#print("Total Price:", total)
#price = 500
#tax = price * 0.18
#total = price + tax
#print("Total Price:", total)
#Code with a function calculate_total(price) that can be reused for multiple price inputs.
def calculate_total(price):
    tax = price * 0.18
    total = price + tax
    return total
print("Total Price:", calculate_total(250))
print("Total Price:", calculate_total(500))
```

OUTPUT:

```
Total Price: 295.0
Total Price: 590.0
```

Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

Task: Use AI to identify and replace all hardcoded “magic numbers” in the code with named constants.

Instructions:

- Create constants at the top of the file.
- Replace all hardcoded occurrences in calculations with these constants.
- Ensure the code remains functional and is easier to maintain.

CODE:

```
# Legacy script with hardcoded values
#print("Area of Circle:", 3.14159 * (7 ** 2))
#print("Circumference of Circle:", 2 * 3.14159 * 7)
#Code with constants like PI = 3.14159 and RADIUS = 7 used in calculations.
PI = 3.14159
RADIUS = 7
print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)
```

OUTPUT:

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

Task: Use AI to improve readability by renaming unclear variables and adding inline comments.

Instructions:

- Replace vague names with meaningful ones.
- Add comments where logic is not obvious. □ Keep functionality exactly the same.

CODE:

```
# Legacy script with poor variable names
#a = 10
#b = 20
#c = a * b / 2
#print(c)
#Code with descriptive variable names like base,height, area_of_triangle,
#plus explanatory comments.
base = 10
height = 20
area_of_triangle = base * height / 2
print("Area of Triangle:", area_of_triangle)
```

OUTPUT:

```
Area of Triangle: 100.0
```

Task Description #6 (Refactoring – Removing Redundant Conditional Logic)

Task: Use AI to refactor a Python script that contains repeated if–else logic for grading students.

Instructions:

- Ask AI to identify redundant conditional checks.
- Replace them with a reusable function.
- Ensure grading logic remains unchanged.

CODE:

```
#marks = 85
#if marks >= 90:
#    print("Grade A")
#elif marks >= 75:
#    print("Grade B")
#else:
#    print("Grade C")
#marks = 72
#if marks >= 90:
#    print("Grade A")
#elif marks >= 75:
#    print("Grade B")
#else:
#    print("Grade C")
#A reusable function like calculate_grade(marks) with clean logic and docstring.
def calculate_grade(marks):
    """
    Calculate the grade based on the marks.
    Parameters:
    marks (int): The marks obtained by the student.
    Returns:
    str: The grade corresponding to the marks.
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"
print("calculate_grade(85):", calculate_grade(85))
print("calculate_grade(72):", calculate_grade(72))
```

OUTPUT:

```
calculate_grade(85): Grade B
calculate_grade(72): Grade C
```

Task Description #7 (Refactoring – Converting Procedural Code to Functions)

Task: Use AI to refactor procedural input–processing logic into functions**.**

Instructions:

- Identify input, processing, and output sections.
- Convert each into a separate function.
- Improve code readability without changing behavior

CODE:

```
#num = int(input("Enter number: "))
#square = num * num
#print("Square:", square)
#Modular code using functions like get_input(), calculate_square(),and display_result().
def get_input():
    return int(input("Enter number: "))
def calculate_square(num):
    return num * num
def display_result(square):
    print("Square:", square)
number = get_input()
result = calculate_square(number)
display_result(result)
```

OUTPUT:

```
Enter number: 25
Square: 625
```

Task Description #8 (Refactoring – Optimizing List Processing)

Task: Use AI to refactor inefficient list processing logic.

Instructions:

- Ask AI to replace manual loops with list comprehensions or built-in functions.
- Ensure output remains identical.

CODE:

```
#nums = [1, 2, 3, 4, 5]
#squares = []
#for n in nums:
#    squares.append(n * n)
#print(squares)
#Optimized version using list comprehension with improved readability

nums = [1, 2, 3, 4, 5]
squares = [n * n for n in nums]
print("Squares of", nums, "are:", squares)
```

OUTPUT:

```
Squares of [1, 2, 3, 4, 5] are: [1, 4, 9, 16, 25]
```